

Secret Benchmark Design

Benchmark Goals

The benchmark was designed with the following criteria in mind:

- **Our cache is large:** the final best-performing cache is 64 sets, 2 ways, and 16 words per line, allowing a total of 2,048 instructions. Therefore, the benchmark should highlight that our processor is performant even with a large benchmark program loop, with close to this many number of instructions, demonstrating advantages over teams who may have implemented a smaller cache.
- **Our cache is associative.** If possible, the benchmark should be designed to highlight how the associativity reduces conflict misses, such as in situations where the benchmark main loop jumps around in the program file instead of staying in a sequential set of instructions.
- **We have data memory lookahead** for memory read instructions, so the benchmark should include load instructions (such as using a lookup table).
- It would be nice to have a benchmark that does a task that could be useful in the real-world.

Benchmark Design

The final benchmark is an iteration of the initial polynomial numerical integration used for preliminary testing. It loops through integrating 19 different 2-D quadratic polynomials sequentially for each x and y coordinate in a 48x48 grid. It also adds an additional "sinusoidal" term to each polynomial based on a mock sin() lookup table, to use additional load instructions. An example of the core code for one of the 19 polynomials is below:

```
uint32_t x2 = x*x;
uint32_t y2 = y*y;
uint32_t xy = x*y;
uint32_t z = 3*x2 + 11*y2 + 8*xy + 6*x + 32*y + 1497;
int32_t t = (11*(int32_t)x + 7*(int32_t)y + 15) * sin_val;
z += (uint32_t)t;
__asm__ volatile ("" : "+r"(z));
return z;
```

This benchmark is large, as intended. The main run_workload loop function is more than 400 instructions long, and the 19 polynomial calculations total to around 1300 instructions (around 69 instructions per polynomial), for a total of 1700 instructions. This is enough to fit in our 2048 instruction cache, but not fit in a smaller 1024 instruction cache. (The number of polynomials was based on testing to find the largest number that could fit in our cache without conflict misses).

The benchmark also generates a large number of conflict misses for a non-associative cache. This is because due to the length of the polynomial function definitions at the start of the file (there are 64 polynomial definitions included on purpose, even though only 19 are used), **the addresses of instructions within the functions to calculate the 4th through the 11th polynomials are made to be exactly $0x4000 = 4096$ instructions apart from the main `run_workload()` function.** Therefore, any direct-mapped cache 4096 instructions or smaller would have conflict misses even if there is still capacity in the cache. But a two-way cache would be able to keep both the polynomial instruction and `run_workload` instruction in its two separate ways.

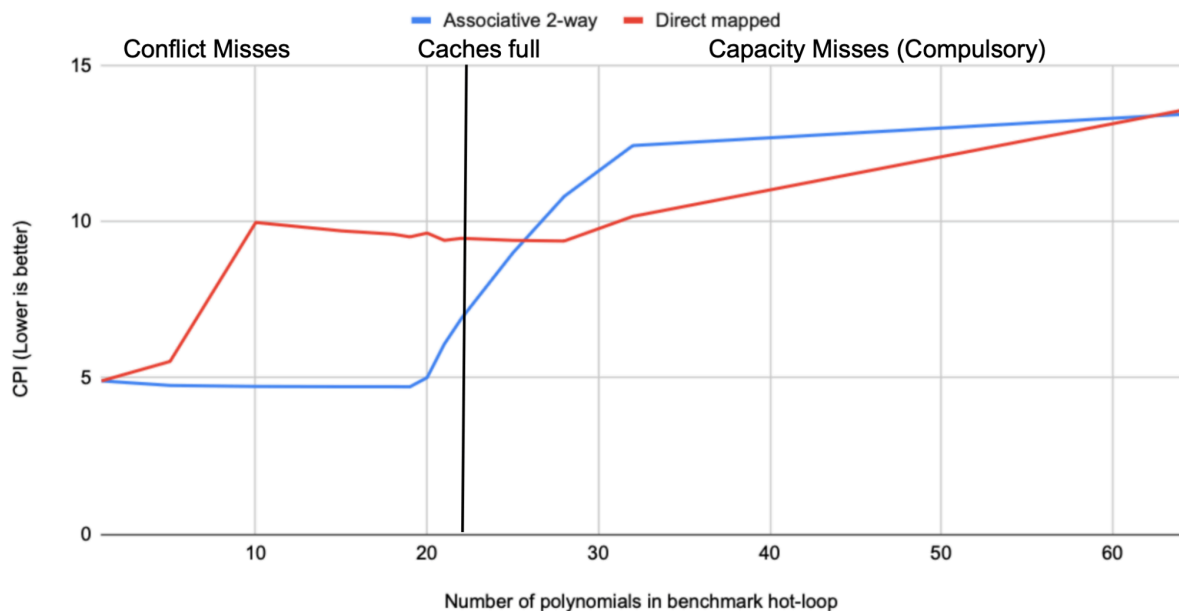
Since the benchmark uses a sine LUT (and other parts of the polynomial calculation logic uses load instructions too), more than a quarter of instructions are memory loads, allowing our data memory lookahead interface to improve CPI by around 0.26.

Effect of Hot-Loop Size and Associativity

The figure below shows that before the hot-loop instruction count exceeds the cache size (at around 22 polynomials), the associative cache indeed outperforms the direct-mapped cache, and this is clearly due to avoided conflict misses. Our benchmark, at 19 polynomials, is done at the "sweet-spot" - the largest instruction count supported by the associative cache before noticeable conflict misses.

Associative vs Direct-Mapped Cache on Benchmark

Both caches have the same size, 2048 total instructions



Interestingly, the direct mapped cache outperforms the associative cache for when the number of instructions exceeds the cache capacity. This is a little surprising but definitely not impossible (for an example, consider 8-line direct mapped vs 4x2-way cache for instruction stream 0, 4, 8, 0, 4, 8...).

Results

As intended, our benchmark demonstrates the advantage of our full configuration - removing any of the three aforementioned elements (large size, associativity, data lookahead) noticeably deteriorates performance.

Configuration	CPI
64x2x16 Cache, Associative, Data Lookahead	4.7135
Same, without associativity (128x16 cache)	9.5077
Same, without data lookahead	4.9738
Same, but cache half the size (32x2x16)	13.4983
Baseline picorv32, no cache, no lookahead	10.1860

Our full configuration outperforms the baseline design with no cache and no datalookahead whatsoever, which achieves 10.1860 CPI, by a factor of 2.161x.

If a team isn't using the fast QDDR SPI flash, and has a non-associative cache or smaller cache, CPI skyrockets to around **39-69**! Our benchmark file doesn't enable QDDR SPI flash, but our processor turns it on in the hardware (register reset value in spimemio.v)

Note that the cache-too-small case achieves worse CPI of around 13.5 than no cache at all, likely due to the high words-per-line requiring sometimes-unused fills. But since all teams are building caches and we have the largest possible cache in BRAM, I expect any team's cache-oriented benchmark should fit in our cache, so I'm not too worried about this case.